School of Computer Science and Engineering

Fall Semester 2026-27

BCSE412L Parallel Computing

SLOT: C1+TC1 **Maximum Marks: 10**

Case Study Assessment

This document consists of 15 case study problem sets. Students can form a batch of size 3 (max.) and choose any one problem statement. Get the prior approval from the course faculty for the problem statement and approval will be given based on the first come first serve model.

Each problem statement has 3 distinct sub-problems and each student in the batch will own a one, but the results should be designed to combine into a single comparative research report. The case study has been formulated based on the following research challenges noticed in the parallel computing.

- How does workload regularity influence the choice of parallel decomposition and scheduling strategy?
- How does task granularity affect speedup, efficiency, and scalability?
- When does communication or synchronization become the dominant performance bottleneck?
- When does GPU/heterogeneous execution outperform CPU-only execution?
- How do task-based and DAG-based approaches behave under heterogeneous resources?
- How do energy, resilience, and resource constraints change the definition of an optimal parallel solution?
- Can workload-aware or learning-based scheduling improve over fixed scheduling policies?

Every student must follow these common requirements unless the assigned case explicitly says otherwise.

- Literature review: minimum 5 scholarly papers; at least 3 should be recent unless the topic requires seminal work.
- Implement or reproduce a clearly defined baseline before optimization.
- Use at least three problem sizes or workload levels.
- Use at least three thread/resource configurations where technically meaningful.
- Report execution time, speedup, efficiency, and scalability whenever applicable.
- Separate computation, communication, synchronization, scheduling, and data-movement overhead where possible.
- Repeat timing experiments sufficiently to reduce noise and report an appropriate summary such as mean/median and variation.
- Include at least three graphs and one summary table.
- Explain the observed performance rather than merely presenting measurements.
- End with limitations, threats to validity, and at least three future research directions.

Few commonly used metrics for assessment:

Metric	Meaning	Required use
Execution time	Time to complete the workload	All case studies
Speedup	Baseline time / parallel time	All applicable studies
Efficiency	Speedup / number of processing resources	All scaling studies
Scalability	Performance as resources/problem size change	All studies
Overhead	Time/cost attributable to parallel execution	Where measurable
Communication time	Data movement between processes/devices	MPI/distributed/GPU studies

Synchronization time	Waiting/coordination cost	Shared-memory/task studies
Energy	Energy consumed during execution	

The Case Study Outcome should be

- Individual member technical note: 2–3 pages.
- Batch report: 8–12 pages integrating the three student contributions.
- Source code with necessary instructions.
- Raw experimental data in CSV format.
- Plots in editable/source form.

List of Assessments

CS1: Task Granularity and Parallel Scalability

S01 — Fine vs. Coarse-Grained Parallelism

Problem statement: Investigate how task size changes scheduling overhead, parallelism, and scalability.

Research question: Is finer-grained parallelism always beneficial?

Objectives:

- Implement coarse, medium, and fine task decompositions.
- Vary threads and task counts.
- Identify the performance crossover point.

Datasets/benchmarks: Synthetic workload; matrix multiplication or vector processing.

Tools/platform: C/C++, OpenMP

Required metrics: Execution time; Speedup; Efficiency; Scheduling overhead

Expected graphs: Execution time vs task size; Speedup vs threads; Efficiency vs granularity

S02 — Granularity under Irregular Workloads

Problem statement: Determine how execution-time variability changes the best task granularity.

Research question: How does workload irregularity change the optimal granularity?

Objectives:

- Generate low, medium, and high variance task distributions.
- Compare several granularities.
- Quantify idle time and imbalance.

Datasets/benchmarks: Synthetic irregular workload; prime search/Mandelbrot-style workload.

Tools/platform: C/C++, OpenMP

Required metrics: Load imbalance; Execution time; Speedup; Idle time

Expected graphs: Workload variance vs time; Imbalance vs threads; Granularity vs efficiency

S03 — Adaptive Task Granularity

Problem statement: Compare fixed and adaptive task sizes when workload characteristics change during execution.

Research question: Can adaptive task sizing improve performance over a fixed task size?

Objectives:

- Implement fixed and adaptive strategies.
- Change workload characteristics during execution.
- Measure adaptation overhead.

Datasets/benchmarks: Synthetic phase-changing workload.

Tools/platform: C/C++, OpenMP tasks

Required metrics: Makespan; Overhead; Speedup; Adaptation cost

Expected graphs: Fixed vs adaptive time; Adaptation overhead; Speedup vs workload change

CS 2: Advanced Scheduling and Load Balancing

S01 — Static vs Dynamic Scheduling

Problem statement: Compare static, dynamic, and guided scheduling for regular and irregular loops.

Research question: Under what workload conditions does dynamic scheduling outperform static scheduling?

Objectives:

- Create controlled workload irregularity.
- Vary thread count.
- Compare scheduling policies.

Datasets/benchmarks: Prime search, Mandelbrot, or irregular loop benchmark.

Tools/platform: C/C++, OpenMP

Required metrics: Execution time; Speedup; Efficiency; Scheduling overhead

Expected graphs: Time vs threads; Speedup vs threads; Overhead vs irregularity

S02 — Scheduling Chunk-Size Optimization

Problem statement: Determine how dynamic/guided chunk size affects performance.

Research question: What chunk size gives the best trade-off between load balance and scheduling overhead?

Objectives:

- Sweep chunk sizes.
- Repeat for several workload sizes.
- Identify robust rather than single-point optimum.

Datasets/benchmarks: Irregular loop benchmark.

Tools/platform: C/C++, OpenMP

Required metrics: Time; Overhead; Imbalance; Efficiency

Expected graphs: Time vs chunk size; Imbalance vs chunk size; Efficiency vs chunk size

S03 — Workload-Aware Scheduling

Problem statement: Develop a rule-based scheduler that selects among static, dynamic, and guided policies based on workload characteristics.

Research question: Can workload-aware policy selection outperform a fixed scheduling policy?

Objectives:

- Characterize regularity before execution.
- Select policy using simple rules.
- Compare against best fixed policy.

Datasets/benchmarks: Synthetic workloads with controlled variance/skew.

Tools/platform: C/C++, OpenMP; Python for analysis

Required metrics: Makespan; Speedup; Decision overhead; Robustness

Expected graphs: Policy vs workload variance; Time comparison; Decision accuracy/benefit

CS3: DAG and Task-Based Parallelism

S01 — Critical-Path-Aware Scheduling

Problem statement: Study scheduling of dependent tasks using critical-path priorities.

Research question: Can critical-path awareness reduce DAG makespan?

Objectives:

- Generate DAGs with different depths and widths.
- Compare priority policies.
- Measure utilization.

Datasets/benchmarks: Synthetic DAG benchmark.

Tools/platform: C++ task runtime/OpenMP tasks/Taskflow

Required metrics: Makespan; Critical-path delay; Utilization; Speedup

Expected graphs: DAG size vs makespan; Processor count vs speedup; Utilization vs DAG width

S02 — Static vs Dynamic DAG Scheduling

Problem statement: Compare precomputed task mappings with runtime task scheduling when task durations are uncertain.

Research question: When does dynamic DAG scheduling justify its runtime overhead?

Objectives:

- Vary execution-time uncertainty.
- Compare static and dynamic execution.
- Separate scheduling overhead.

Datasets/benchmarks: Synthetic DAGs with known/unknown task costs.

Tools/platform: C++ task runtime

Required metrics: Makespan; Scheduling overhead; Idle time; Robustness

Expected graphs: Uncertainty vs makespan; Overhead vs task count; Speedup vs resources

S03 — Heterogeneous DAG Scheduling

Problem statement: Map dependent tasks to CPU and GPU resources according to task characteristics.

Research question: How should DAG tasks be assigned to heterogeneous resources?

Objectives:

- Classify tasks as CPU/GPU-favorable.

- Compare CPU-only, GPU-only, and heterogeneous mapping.
- Measure data-transfer cost.

Datasets/benchmarks: Synthetic heterogeneous DAG; matrix/image kernels.

Tools/platform: CUDA + C++ task runtime

Required metrics: Makespan; GPU utilization; Transfer time; Speedup

Expected graphs: CPU/GPU assignment vs time; Transfer vs computation; Utilization vs DAG size

CS 4: Parallel Graph Processing

S01 — Parallel Breadth-First Search

Problem statement: Compare sequential and parallel BFS implementations on graphs of increasing size.

Research question: How does graph size and structure affect BFS scalability?

Objectives:

- Use multiple graph structures.
- Compare CPU parallel and GPU if available.
- Measure traversal throughput.

Datasets/benchmarks: SNAP/SuiteSparse/Graph500-style graphs.

Tools/platform: C++ OpenMP/CUDA

Required metrics: Traversal time; Speedup; TEPS/throughput; Scalability

Expected graphs: Time vs graph size; Speedup vs threads; Throughput vs density

S02 — Parallel PageRank

Problem statement: Study iterative PageRank execution on graphs with different sizes and sparsity.

Research question: How do graph size and sparsity affect parallel PageRank performance?

Objectives:

- Vary graph size and sparsity.
- Compare CPU and GPU where feasible.
- Measure convergence and runtime.

Datasets/benchmarks: SNAP/SuiteSparse graphs.

Tools/platform: C++ OpenMP/CUDA

Required metrics: Iteration time; Total time; Speedup; Throughput

Expected graphs: Time vs vertices; Time vs edges; Speedup vs resources

S03 — Graph Structure vs Parallel Performance

Problem statement: Determine how topology affects the scalability of a common graph kernel.

Research question: Which graph characteristics predict parallel performance?

Objectives:

- Compare sparse, dense, random, and scale-free graphs.
- Keep algorithm constant.

- Relate structural statistics to performance.

Datasets/benchmarks: SNAP and synthetic graph families.

Tools/platform: C++ OpenMP

Required metrics: Execution time; Speedup; Degree distribution; Memory use

Expected graphs: Graph density vs time; Degree statistics vs speedup; Memory vs graph size

CS 5: Communication Optimization

S01 — Collective Communication Performance

Problem statement: Measure the scalability of common MPI collective operations.

Research question: How do message size and process count affect collective communication?

Objectives:

- Benchmark broadcast, scatter, gather, reduce, and all-reduce.
- Vary message sizes and process counts.
- Compare measured trends.

Datasets/benchmarks: Synthetic message workloads.

Tools/platform: MPI

Required metrics: Latency; Bandwidth; Communication time; Scalability

Expected graphs: Time vs message size; Time vs processes; Effective bandwidth

S02 — Overlapping Communication and Computation

Problem statement: Compare blocking execution with communication/computation overlap.

Research question: How much communication latency can be hidden by overlap?

Objectives:

- Construct balanced and communication-heavy workloads.
- Measure blocking and nonblocking versions.
- Separate useful work and waiting.

Datasets/benchmarks: MPI synthetic stencil/halo-exchange workload.

Tools/platform: MPI

Required metrics: Total time; Communication time; Overlap benefit; Speedup

Expected graphs: Time decomposition; Overlap benefit vs message size; Speedup vs processes

S03 — Topology-Aware Communication

Problem statement: Compare ring, tree, and hierarchical communication patterns.

Research question: How does communication topology affect scalability?

Objectives:

- Implement equivalent communication patterns.
- Vary process count and message size.
- Compare latency and throughput.

Datasets/benchmarks: Synthetic communication benchmark.

Tools/platform: MPI

Required metrics: Latency; Bandwidth; Messages/sec; Scalability

Expected graphs: Latency vs processes; Bandwidth vs message size; Topology comparison

CS 6: MPI + OpenMP Hybrid Computing

S01 — MPI vs OpenMP vs Hybrid

Problem statement: Compare three programming approaches on the same numerical workload.

Research question: Which programming model is best under different problem sizes?

Objectives:

- Implement equivalent versions.
- Use controlled resource counts.
- Compare scaling.

Datasets/benchmarks: Matrix multiplication or stencil.

Tools/platform: MPI, OpenMP, C/C++

Required metrics: Execution time; Speedup; Efficiency; Memory

Expected graphs: Time vs resources; Speedup comparison; Efficiency

S02 — Hybrid Process/Thread Mapping

Problem statement: Investigate how MPI process count and OpenMP thread count should be combined.

Research question: What MPI×thread configuration minimizes runtime on a fixed resource budget?

Objectives:

- Keep total cores approximately constant.
- Sweep process/thread combinations.
- Measure communication and computation.

Datasets/benchmarks: Matrix or stencil benchmark.

Tools/platform: MPI + OpenMP

Required metrics: Time; Communication time; Speedup; Efficiency

Expected graphs: Time heatmap of MPI×threads; Communication vs configuration; Speedup

Final-paper subsection: 8.2 Hierarchical Parallelism

S03 — Strong and Weak Scaling of Hybrid Programs

Problem statement: Evaluate strong and weak scaling of a hybrid application.

Research question: How does hybrid execution scale as resources and workload increase?

Objectives:

- Conduct strong-scaling and weak-scaling experiments.
- Identify saturation points.

Datasets/benchmarks: Distributed stencil/matrix benchmark.

Tools/platform: MPI + OpenMP

Required metrics: Strong scaling; Weak scaling; Efficiency; Communication

Expected graphs: Strong scaling curve; Weak scaling curve; Efficiency

Final-paper subsection: 8.3 Scalability of Hybrid Parallel Programs

CS 7: CPU–GPU Heterogeneous Computing

S01 — CPU vs GPU Performance

Problem statement: Compare multicore CPU and GPU execution of a computational kernel.

Research question: For which workloads and sizes does GPU execution provide meaningful benefit?

Objectives:

- Vary problem size.
- Measure CPU and GPU separately.
- Include transfer time.

Datasets/benchmarks: Matrix multiplication/vector operations.

Tools/platform: C++ OpenMP + CUDA

Required metrics: Execution time; Speedup; Throughput; Transfer time

Expected graphs: Time vs size; Speedup vs size; Transfer fraction

S02 — GPU Offloading Crossover Point

Problem statement: Find the problem size at which GPU execution becomes faster than CPU execution.

Research question: At what problem size does GPU offloading become beneficial?

Objectives:

- Use logarithmically increasing sizes.
- Separate kernel and end-to-end time.
- Identify crossover.

Datasets/benchmarks: Matrix/vector/image kernel.

Tools/platform: CUDA/OpenMP offload

Required metrics: Kernel time; End-to-end time; Speedup; Transfer overhead

Expected graphs: Problem size vs time; Speedup vs size; Transfer fraction

S03 — CPU–GPU Workload Partitioning

Problem statement: Split a workload between CPU and GPU and compare with CPU-only/GPU-only execution.

Research question: Is CPU+GPU execution always better than GPU-only execution?

Objectives:

- Vary CPU/GPU workload fraction.
- Measure transfer and synchronization costs.
- Find optimal split.

Datasets/benchmarks: Independent-task workload such as image tiles.

Tools/platform: CUDA + OpenMP

Required metrics: Makespan; GPU utilization; CPU utilization; Speedup

Expected graphs: Split ratio vs time; Utilization vs split; Speedup

CS 8: GPU Memory and Kernel Optimization

S01 — GPU Memory Hierarchy

Problem statement: Compare alternative memory-access strategies for a CUDA kernel.

Research question: How strongly does memory placement affect GPU performance?

Objectives:

- Compare global/shared/appropriate read-only strategies.
- Control arithmetic work.
- Measure memory effects.

Datasets/benchmarks: Tiled matrix operation/convolution.

Tools/platform: CUDA, Nsight Compute

Required metrics: Kernel time; Memory throughput; Occupancy

Expected graphs: Time vs matrix size; Bandwidth vs strategy; Occupancy vs block size

S02 — CUDA Thread-Block Configuration

Problem statement: Study the effect of grid and block configuration on kernel performance.

Research question: What configuration provides robust performance across workload sizes?

Objectives:

- Sweep block sizes.
- Measure occupancy and execution time.
- Compare several problem sizes.

Datasets/benchmarks: CUDA matrix/vector/convolution kernel.

Tools/platform: CUDA, Nsight Compute

Required metrics: Kernel time; Occupancy; Throughput

Expected graphs: Time vs block size; Occupancy vs block size; Throughput

S03 — GPU Data-Movement Bottlenecks

Problem statement: Separate host-device transfers from kernel execution.

Research question: When does data movement dominate the benefit of GPU acceleration?

Objectives:

- Measure H→D, kernel, D→H separately.
- Vary data size.
- Compare end-to-end speedup.

Datasets/benchmarks: CUDA image/matrix workload.

Tools/platform: CUDA, Nsight Systems

Required metrics: Transfer time; Kernel time; End-to-end time; Speedup

Expected graphs: Time decomposition; Transfer fraction vs size; Speedup

CS 9: Parallel Image Processing

S01 — Parallel Image Filtering

Problem statement: Compare sequential, OpenMP, and GPU image filtering.

Research question: How does image resolution affect filtering scalability?

Objectives:

- Use multiple resolutions.
- Compare CPU and GPU.
- Measure throughput.

Datasets/benchmarks: Standard image collections; instructor-provided images.

Tools/platform: C++ OpenMP/CUDA

Required metrics: Time; Speedup; Images/sec; Memory

Expected graphs: Time vs resolution; Speedup vs resolution; Throughput

S02 — Parallel Edge Detection

Problem statement: Parallelize edge detection and study scaling with image resolution.

Research question: How does image size influence scalability and memory behavior?

Objectives:

- Vary resolution and thread count.
- Compare CPU and GPU if available.

Datasets/benchmarks: Standard image collection.

Tools/platform: OpenMP/CUDA

Required metrics: Time; Speedup; Throughput; Memory

Expected graphs: Time vs pixels; Speedup vs threads; Throughput

S03 — GPU Image Convolution

Problem statement: Compare CPU and GPU convolution under varying kernel and image sizes.

Research question: How do kernel size and image resolution influence GPU advantage?

Objectives:

- Vary image and kernel dimensions.
- Separate transfer and computation.

Datasets/benchmarks: Standard image collection.

Tools/platform: CUDA + C++

Required metrics: Kernel time; End-to-end time; Speedup; Bandwidth

Expected graphs: Speedup vs kernel size; Time vs image size; Transfer fraction

CS 10: Parallel Machine Learning

S01 — Parallel K-means

Problem statement: Evaluate parallel K-means with increasing data and processor counts.

Research question: How does dataset size affect parallel clustering scalability?

Objectives:

- Vary points and clusters.
- Compare sequential and parallel.
- Measure convergence.

Datasets/benchmarks: MNIST/UCI/synthetic datasets.

Tools/platform: C++ OpenMP or Python multiprocessing

Required metrics: Training time; Iterations; Speedup; Efficiency

Expected graphs: Time vs samples; Speedup vs threads; Iterations vs configuration

S02 — Parallel k-NN Search

Problem statement: Study parallel nearest-neighbor search on increasing datasets.

Research question: How does dataset size affect parallel search efficiency?

Objectives:

- Vary training size and query count.
- Compare CPU parallel strategies.
- Measure accuracy separately from runtime.

Datasets/benchmarks: MNIST/UCI datasets.

Tools/platform: C++ OpenMP / Python

Required metrics: Search time; Queries/sec; Speedup; Accuracy

Expected graphs: Time vs dataset size; Throughput vs threads; Speedup

S03 — Parallel Data Preprocessing

Problem statement: Measure the benefit of parallelizing normalization and feature-preparation pipelines.

Research question: Which preprocessing stages scale well and which remain bottlenecks?

Objectives:

- Profile each stage.
- Parallelize independent stages.
- Compare pipeline and stage-level parallelism.

Datasets/benchmarks: MNIST/CIFAR-10/UCI.

Tools/platform: Python + multiprocessing/NumPy/OpenMP where appropriate

Required metrics: Stage time; Pipeline time; Speedup; CPU utilization

Expected graphs: Stage contribution; Speedup vs workers; Pipeline throughput

CS 11: Distributed Deep Learning

S01 — Data-Parallel Deep Learning

Problem statement: Compare single-GPU and multi-GPU data-parallel training.

Research question: How does distributed data parallelism scale with GPU count?

Objectives:

- Use fixed training configuration.
- Measure throughput and scaling.
- Track communication.

Datasets/benchmarks: CIFAR-10.

Tools/platform: PyTorch Distributed

Required metrics: Training time; Samples/sec; Scaling efficiency; Communication time

Expected graphs: Throughput vs GPUs; Efficiency vs GPUs; Communication fraction

S02 — All-Reduce Communication Bottleneck

Problem statement: Quantify the communication cost of distributed gradient synchronization.

Research question: At what scale does all-reduce become a dominant bottleneck?

Objectives:

- Measure communication separately.
- Vary batch size and GPU count.
- Compare computation/communication ratio.

Datasets/benchmarks: CIFAR-10 or another standard vision dataset.

Tools/platform: PyTorch Distributed

Required metrics: All-reduce time; Iteration time; Throughput; Communication fraction

Expected graphs: All-reduce vs GPUs; Communication fraction; Throughput

S03 — Batch Size and Distributed Scalability

Problem statement: Study the interaction between batch size, GPU count, throughput, and convergence.

Research question: How does batch size affect distributed scaling and learning behavior?

Objectives:

- Vary batch size and GPU count.
- Record throughput and validation behavior.

Datasets/benchmarks: CIFAR-10.

Tools/platform: PyTorch Distributed

Required metrics: Throughput; Epoch time; Scaling efficiency; Validation metric

Expected graphs: Throughput vs batch size; Efficiency vs GPUs; Convergence/time

CS 12: Parallel Graph Neural Networks

S01 — Parallel GNN Training

Problem statement: Compare single-device and multi-device GNN training.

Research question: How does graph size affect GNN training scalability?

Objectives:

- Vary graph/dataset size.
- Measure training throughput and memory.

Datasets/benchmarks: Cora, PubMed, Reddit or another accessible benchmark.

Tools/platform: PyTorch Geometric/DGL

Required metrics: Training time; Epoch time; Throughput; Memory

Expected graphs: Time vs graph size; Speedup vs devices; Memory

S02 — Graph Partitioning for Distributed GNNs

Problem statement: Compare graph partitioning strategies for distributed GNN workloads.

Research question: How do edge cuts and partition balance influence communication and training time?

Objectives:

- Compare random and balanced/topology-aware partitioning.
- Measure cut edges and load balance.

Datasets/benchmarks: Cora/PubMed/Reddit or synthetic graphs.

Tools/platform: PyTorch Geometric/DGL + Python

Required metrics: Edge cut; Partition imbalance; Communication; Training time

Expected graphs: Edge cut vs time; Imbalance vs time; Communication vs partitions

S03 — Communication–Computation Trade-off in GNNs

Problem statement: Quantify message-passing communication against local computation.

Research question: When does communication dominate distributed GNN execution?

Objectives:

- Profile computation and communication.
- Vary partitions and graph density.

Datasets/benchmarks: Standard GNN graph benchmarks.

Tools/platform: PyTorch Geometric/DGL

Required metrics: Compute time; Communication time; Iteration time; Scalability

Expected graphs: Compute vs communication; Communication fraction; Scaling

CS 13: Energy-Aware Parallel Computing

S01 — Performance vs Energy

Problem statement: Study whether the fastest configuration is also the most energy-efficient.

Research question: What is the trade-off between execution time and energy?

Objectives:

- Vary thread/resource count.
- Measure power and runtime.
- Compute energy-delay product.

Datasets/benchmarks: Matrix/stencil or CPU benchmark.

Tools/platform: C/C++ OpenMP; Linux power tools where available

Required metrics: Runtime; Power; Energy; EDP

Expected graphs: Energy vs threads; Time vs energy; EDP vs threads

S02 — Frequency Scaling and Parallel Efficiency

Problem statement: Study the interaction of processor frequency and parallelism.

Research question: Can lower frequency provide useful energy savings with acceptable performance loss?

Objectives:

- Vary available frequency states if supported.
- Vary thread count.
- Measure energy/performance.

Datasets/benchmarks: CPU numerical benchmark.

Tools/platform: Linux performance/power tools

Required metrics: Runtime; Energy; Power; Speedup

Expected graphs: Frequency vs runtime; Frequency vs energy; Energy-delay

S03 — Energy-Aware Scheduling

Problem statement: Compare performance-oriented, energy-oriented, and balanced scheduling policies.

Research question: Can energy-aware scheduling reduce energy without unacceptable runtime increase?

Objectives:

- Implement policy alternatives.
- Use several workload intensities.
- Report Pareto trade-offs.

Datasets/benchmarks: OpenMP irregular workload.

Tools/platform: C/C++, OpenMP; Python analysis

Required metrics: Runtime; Energy; EDP; Efficiency

Expected graphs: Pareto frontier; Energy vs runtime; Policy comparison

CS 14: Fault Tolerance and Resilience

S01 — Fault Impact on Parallel Execution

Problem statement: Inject controlled worker/process failures and measure performance impact.

Research question: How does failure rate affect makespan and useful work?

Objectives:

- Inject failures at controlled points.
- Measure recovery/no-recovery cases where possible.

Datasets/benchmarks: MPI synthetic workload.

Tools/platform: MPI + fault-injection scripts

Required metrics: Failure rate; Makespan; Lost work; Recovery time

Expected graphs: Failure rate vs time; Lost work vs failure rate; Scaling

S02 — Checkpointing and Recovery

Problem statement: Measure the overhead and benefit of periodic/adaptive checkpointing.

Research question: What checkpoint interval provides a useful resilience-performance trade-off?

Objectives:

- Compare no checkpoint, periodic, and adaptive checkpointing.
- Inject failures.

Datasets/benchmarks: MPI numerical workload.

Tools/platform: MPI + filesystem/checkpoint mechanism

Required metrics: Checkpoint overhead; Recovery time; Lost work; Total time

Expected graphs: Interval vs overhead; Interval vs lost work; Total time

S03 — Fault-Tolerant Distributed Computation

Problem statement: Study recovery strategies in a distributed parallel application.

Research question: Which recovery strategy minimizes service interruption and recomputation?

Objectives:

- Compare restart/recovery alternatives.
- Measure recovery under multiple failure scenarios.

Datasets/benchmarks: MPI workload.

Tools/platform: MPI + fault-injection framework/scripts

Required metrics: Recovery time; Availability proxy; Lost computation; Overhead

Expected graphs: Failure scenario vs recovery time; Overhead vs resilience; Lost work

CS 15: AI-Based and Adaptive Scheduling

S01 — Evolutionary Task Scheduling

Problem statement: Compare heuristic scheduling with a genetic algorithm for DAG/task workloads.

Research question: Can evolutionary optimization reduce makespan compared with baseline heuristics?

Objectives:

- Define chromosome/fitness representation.
- Compare with FCFS/priority heuristic.
- Vary DAG size.

Datasets/benchmarks: Synthetic DAGs.

Tools/platform: Python + DEAP or equivalent; C++ workload simulator

Required metrics: Makespan; Utilization; Optimization time; Improvement

Expected graphs: DAG size vs makespan; Optimization time; Improvement vs workload

S02 — Reinforcement-Learning-Based Scheduling

Problem statement: Investigate whether an RL policy can learn resource/task assignment.

Research question: Can RL scheduling improve over a conventional heuristic after training?

Objectives:

- Define state/action/reward.
- Train on representative workloads.
- Test on unseen workloads.

Datasets/benchmarks: Synthetic DAG/task environment.

Tools/platform: Python + PyTorch

Required metrics: Makespan; Utilization; Inference overhead; Generalization

Expected graphs: Training reward; Makespan vs method; Generalization performance

S03 — Adaptive Workload-Aware Scheduler

Problem statement: Develop a scheduler that selects or combines policies according to workload characteristics.

Research question: Can an adaptive scheduler outperform a fixed scheduling policy across diverse workloads?

Objectives:

- Use findings from Batches 1–3 and 7.
- Build workload features.
- Compare adaptive vs fixed policies.

Datasets/benchmarks: Synthetic regular/irregular/heterogeneous workloads.

Tools/platform: Python/C++ + OpenMP/CUDA as appropriate

Required metrics: Makespan; Speedup; Decision overhead; Robustness

Expected graphs: Workload feature vs selected policy; Adaptive vs fixed time; Robustness across workloads

Report Format

1. Title

2. Abstract

- 150–200 words summarizing problem, method, main result, and conclusion.

3. Introduction (along with literature survey)

- Problem motivation and why parallel execution matters.
- Minimum 5 scholarly papers; identify a gap, limitation, or unresolved comparison.

4. Problem and Research Question

- Precise problem definition, research question, and hypotheses if appropriate.

6. Methodology

- Baseline, parallel strategy, hardware/software, datasets, parameters, and repetitions.

7. Results and Discussion

- Tables and graphs with concise interpretation.
- Why the observed results occurred; bottlenecks; comparison with literature.

8. Limitations

- Hardware limitations, dataset limitations, measurement noise, or implementation constraints.

9. Conclusion and Future Work

- Answer the research question and propose next steps.

10. References

- Use one consistent scholarly citation style.

Note

- The three students in a batch **must not** submit three disconnected reports. They must build one integrated batch narrative.
- Student 1 establishes the baseline problem and algorithmic behavior. Student 2 investigates the main performance/architecture dimension. Student 3 performs optimization, comparison, or adaptation. Each student must have a clearly identifiable individual contribution.
- The batch report must contain a **common** introduction, common experimental environment, a combined results table, and a comparative discussion.
- The three students must use **compatible data formats** and terminology so that results can later be merged into the class master dataset.
- **If a batch has one or two student(s)**, they can take any one or two problem statements according to the availability.
- Each experiment should produce one CSV row with at least:

Student_ID, Batch_ID, Workload, Dataset, Problem_Size, Algorithm, Parallel_Model, Hardware, Resource_Count, Threads, Execution_Time, Speedup, Efficiency, Communication_Time, Synchronization_Time, Scheduling_Time, Memory_Use, Energy_if_available, Run_ID

- Students should preserve raw measurements. Derived metrics such as speedup and efficiency should be computed from the raw data rather than entered manually.

- Record CPU/GPU model, number of cores, RAM, GPU memory, operating system, compiler/runtime versions, and relevant driver versions.
- Record compiler optimization flags and runtime configuration.
- Provide a README with exact commands used to reproduce the main results.
- Do not report a speedup without identifying the baseline.
- Do not compare results obtained under substantially different hardware without explicitly labeling the comparison.
- Keep raw measurements and scripts used to generate plots.
- Use the same dataset and problem-size definitions across repeated runs.
- Report unexpected results rather than deleting them; explain outliers where possible.

Deadline to submit the document (Word) in VTop: 19 October 2026

*** ALL THE BEST***